

Tree Borrows

Neven Villani,¹ Johannes Hostert,² Derek Dreyer,³ Ralf Jung²

PLDI'25

2025-06-19

¹Univ. Grenoble Alpes, Verimag

²ETH Zurich

³MPI-SWS

Rust's type system enables powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
  
    *x = 13;  
  
    *y = 20;  
  
    *x  
  
}
```

Rust's type system enables powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

mutable thus disjoint

*x is unchanged

Rust's type system enables powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

mutable thus disjoint

*x has known value

*x is unchanged

always returns 13

Rust's type system enables powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
  
    *x = 13;  
  
    *y = 20;  
  
    13 // formerly *x: one fewer load from memory  
  
}
```

Type-level guarantees for references



`&mut` → mutation, no aliasing

`&` → aliasing, no mutation

Escape hatch: **unsafe**

Can use **unchecked operations** to do **low-level manipulations**

```
unsafe {  
  // Code within this block can effectively  
  // bypass some parts of the typechecker.  
  ...  
}
```

Within **unsafe** it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized
- absence of data races
- ...

violations trigger UB
(Undefined Behavior)

What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    *x
}

fn main() {
    let mut root = 42;
    let ptr = &raw mut root;
    let x = unsafe { &mut *ptr };
    let y = unsafe { &mut *ptr };
    println!("{}", write_both(x, y)); // prints 20
}
```


What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

```
fn main() {  
    let mut root = 42;  
    let ptr = &raw mut root;  
    let x = unsafe { &mut *ptr };  
    let y = unsafe { &mut *ptr };  
    println!("{}", write_both(x, y)); // prints 20  
}
```

What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    *x
}

fn main() {
    let mut root = 42;
    let ptr = &raw mut root;
    let x = unsafe { &mut *ptr };
    let y = unsafe { &mut *ptr };
    println!("{}", write_both(x, y)); // prints 20
}
```

What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}  
  
fn main() {  
    let mut root = 42;  
    let ptr = &raw mut root;  
    let x = unsafe { &mut *ptr };  
    let y = unsafe { &mut *ptr };  
    println!("{}", write_both(x, y)); // prints 20  
}
```

What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    13
}

fn main() {
    let mut root = 42;
    let ptr = &raw mut root;
    let x = unsafe { &mut *ptr };
    let y = unsafe { &mut *ptr };
    println!("{}", write_both(x, y)); // prints 13
}
```

What if **unsafe** code is misused ?

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {  
    *x = 13;  
    *y = 20;  
    *x  
}
```

unsafe code can break the assumptions
that optimizations need!

```
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
println!("{}", write_both(x, y)); // prints 20  
}
```

Expanding our notion of UB

Within **unsafe** it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized
- absence of data races
- compliance with aliasing rules **NEW!**

} violations trigger UB

Expanding our notion of UB

Within **unsafe** it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized
- absence of data races
- compliance with aliasing rules ^{NEW!}

} violations trigger UB

Tree Borrows (TB): defines those aliasing rules

Compiler **assumes absence of UB**, exploits this for optimizations

weak optimizations

hard to write correct code



Expanding our notion of UB

Within `unsafe` it is the programmer's responsibility to check

- that pointers are non-null
- that memory is initialized

Sounds familiar?

TB is the successor of **Stacked Borrows**, which has the same purpose.

weak optimizations

hard to write correct code



Stacked Borrows (SB)

[Jung et al., POPL'20]

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

A diagram showing a memory stack. It consists of a single rectangular box with the word "root" inside. The box has a double-line border, with the top line being thicker than the bottom line.

- new stack at root

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

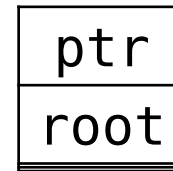
A rectangular box containing the text "root". The box has a double underline at the bottom, indicating it is the current active memory location.

- ✓ root is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

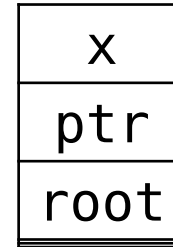


- ✓ root is at the top
- push ptr

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

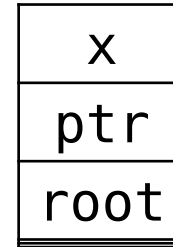


- ✓ ptr is at the top
- push x

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

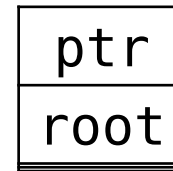


- pop until ptr is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

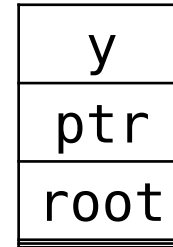


- pop until ptr is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

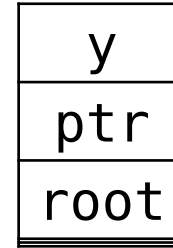


- pop until ptr is at the top
- push y

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```



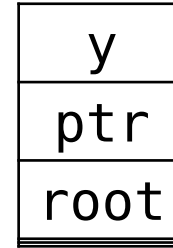
- search for x

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed. Stacked Borrows extends the well-bracketedness to **unsafe**.

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;
```

UB!



Can't use x if it is not in the stack

Desired outcome: UB

SB was **implemented** in Miri (official interpreter and UB detector)

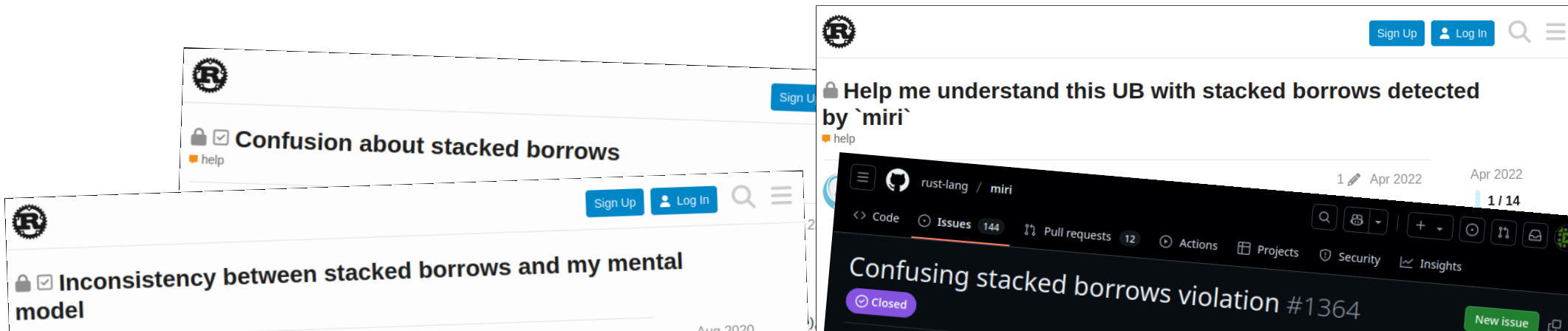
→ included in many projects' CI

→ many bugs detected (e.g. in stdlib)

SB was **implemented** in Miri (official interpreter and UB detector)

→ included in many projects' CI

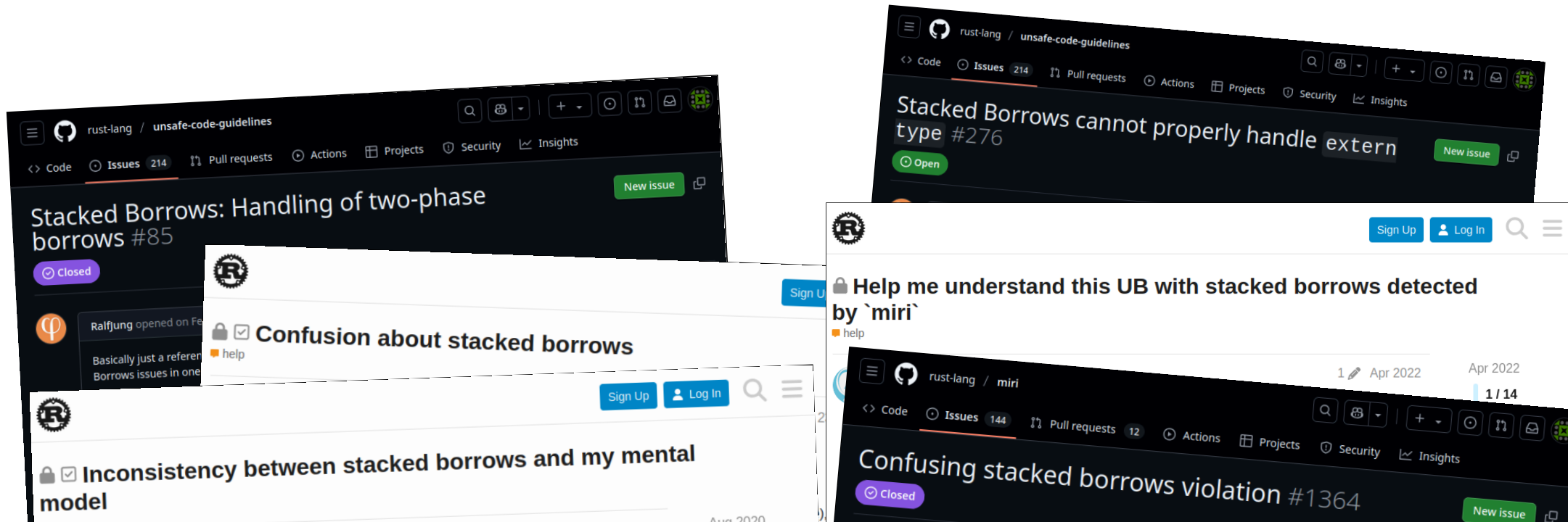
→ many bugs detected (e.g. in stdlib)



SB was **implemented** in Miri (official interpreter and UB detector)

→ included in many projects' CI

→ many bugs detected (e.g. in stdlib)



Anecdotal evidence supported by data:

- analysis of 30 000 libraries
- 6000+ tests have aliasing UB under Stacked Borrows (leading cause of UB)

Tree Borrows uses a **tree** instead of a stack to track borrows

Out of 30 000 most downloaded libraries,
54% fewer tests with aliasing UB when using Tree Borrows

Tree Borrows uses a **tree** instead of a stack to track borrows

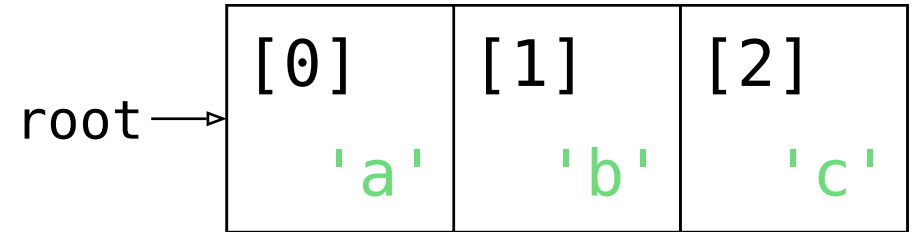
Out of 30 000 most downloaded libraries,
54% fewer tests with aliasing UB when using Tree Borrows

Fixes known technical limitations of SB,
incl. 2-phase borrows, extern types, **pointer offsets**

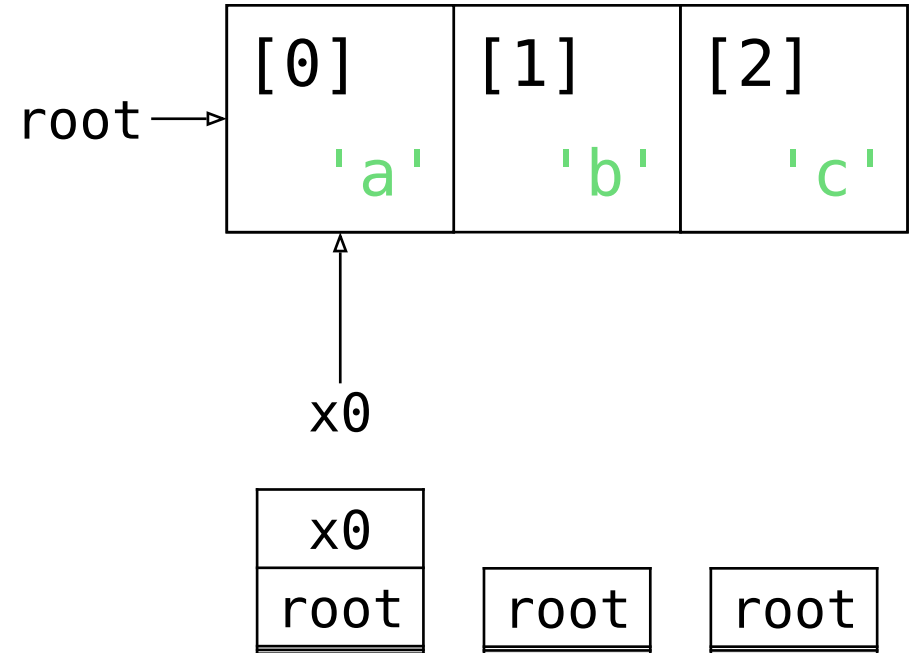
From Stacks to Trees

```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```

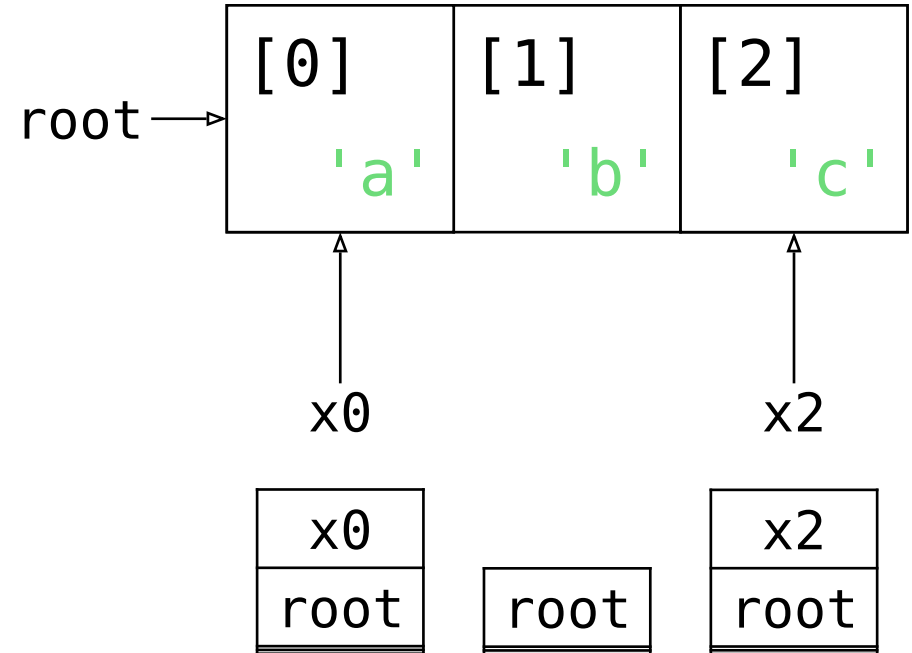
```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```



```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```



```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```

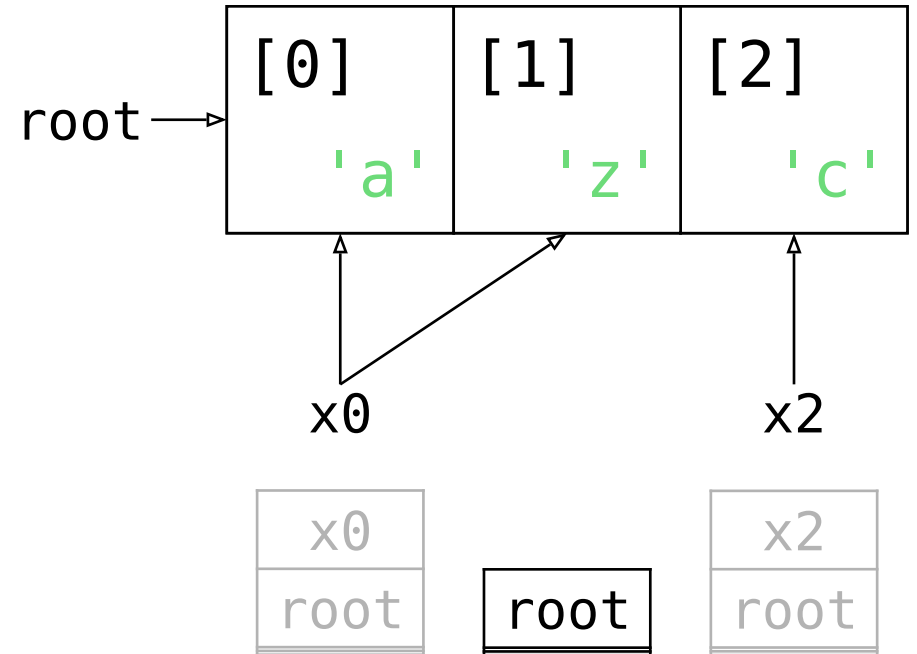


```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```

```
// Scenario 1
```

```
unsafe { *x0.add(1) = 'z'; }
```

Desired outcome: not UB




```

let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];

```

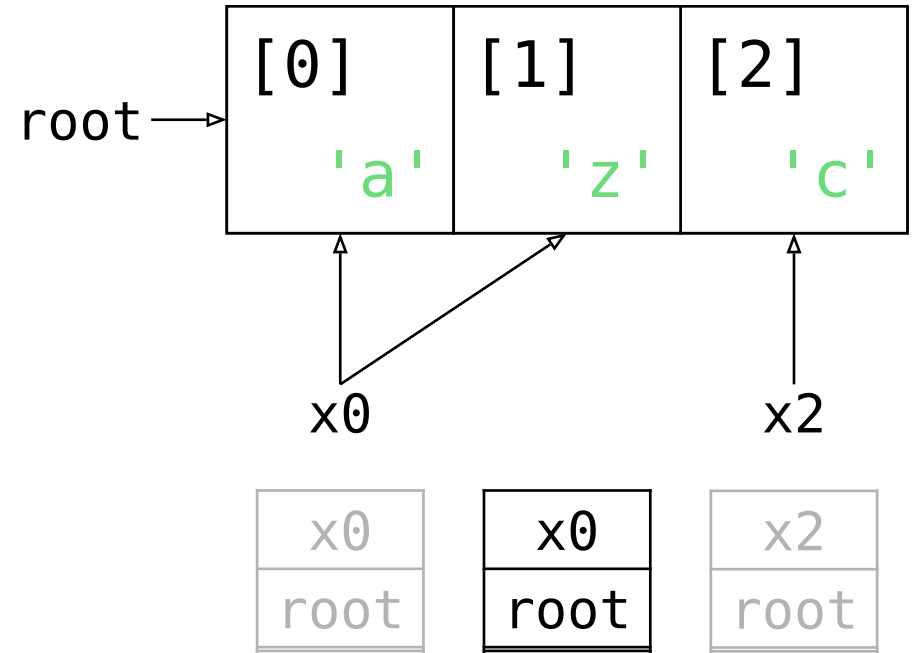
// Scenario 1

```

unsafe { *x0.add(1) = 'z'; }

```

Desired outcome: not UB

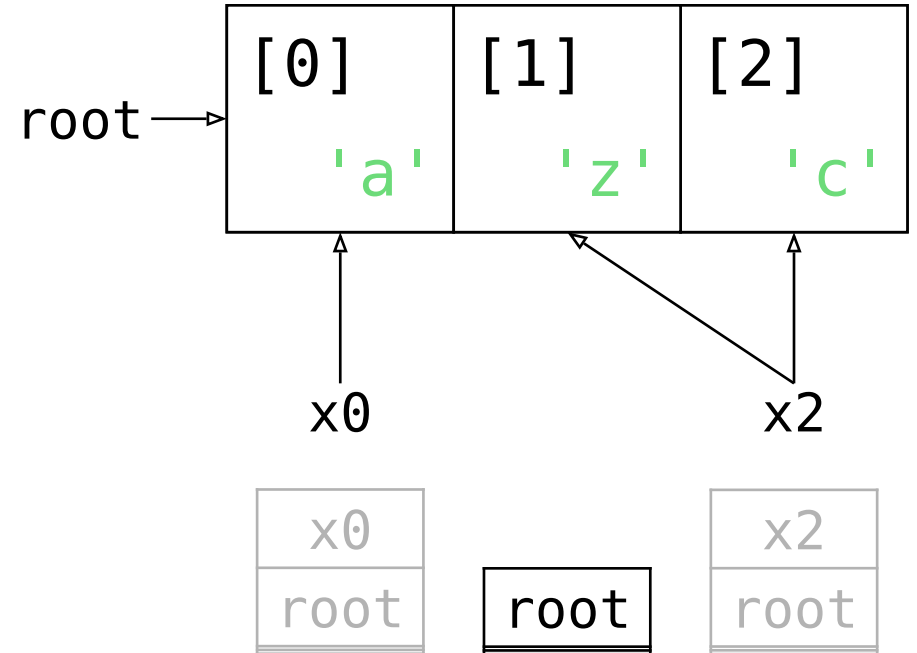


```
let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
```

```
// Scenario 2
```

```
unsafe { *x2.sub(1) = 'z'; }
```

Desired outcome: not UB

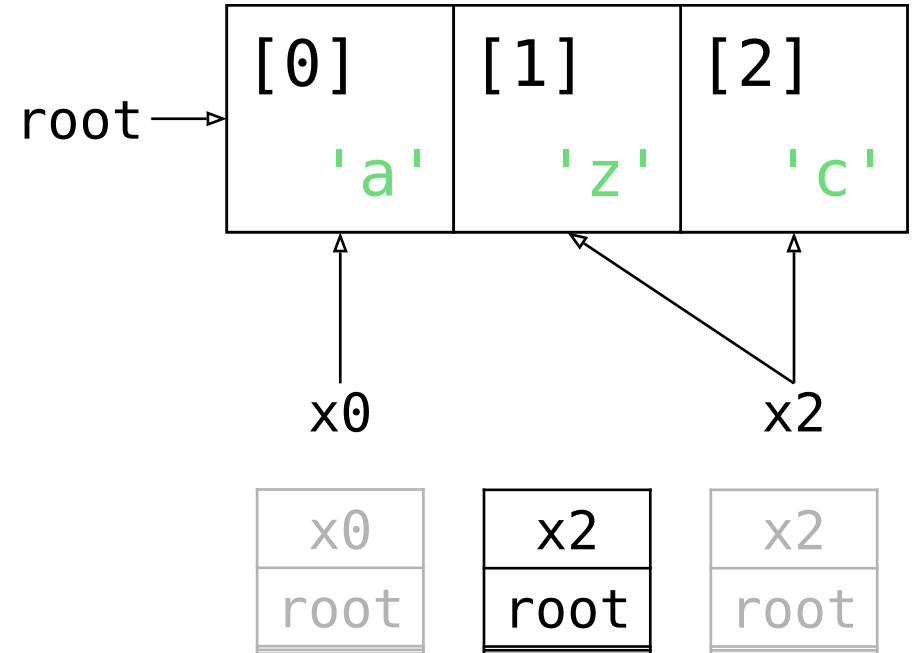


```
let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
```

// Scenario 2

```
unsafe { *x2.sub(1) = 'z'; }
```

Desired outcome: not UB



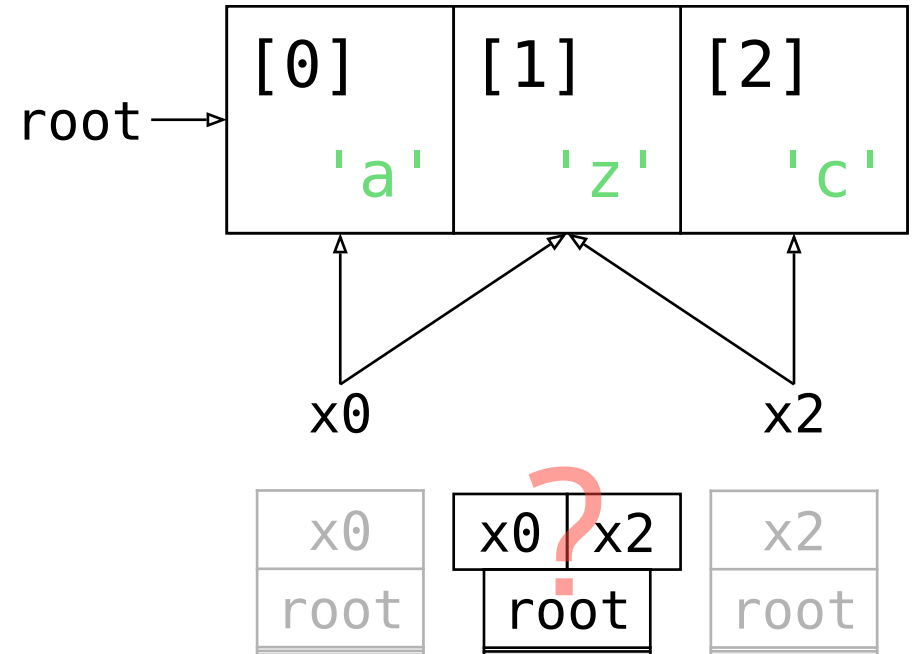
```

let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];

// Scenario 1 or 2
unsafe { *??? = 'z'; }

```

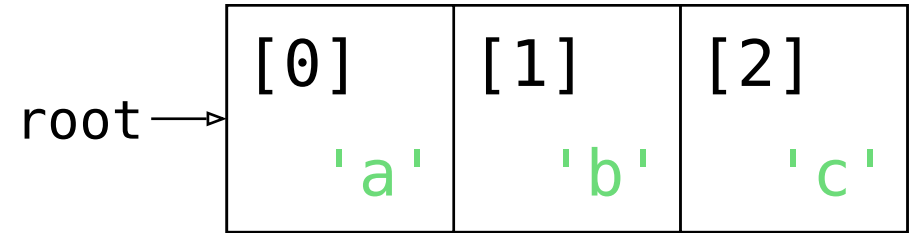
Desired outcome: not UB



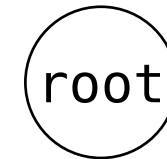
```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```

Desired outcome: not UB

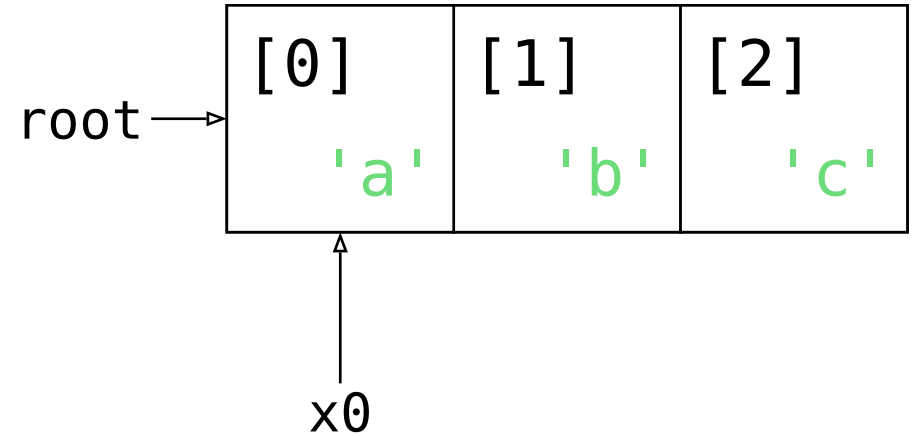
```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```



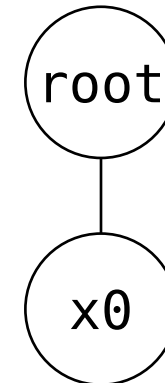
Desired outcome: not UB



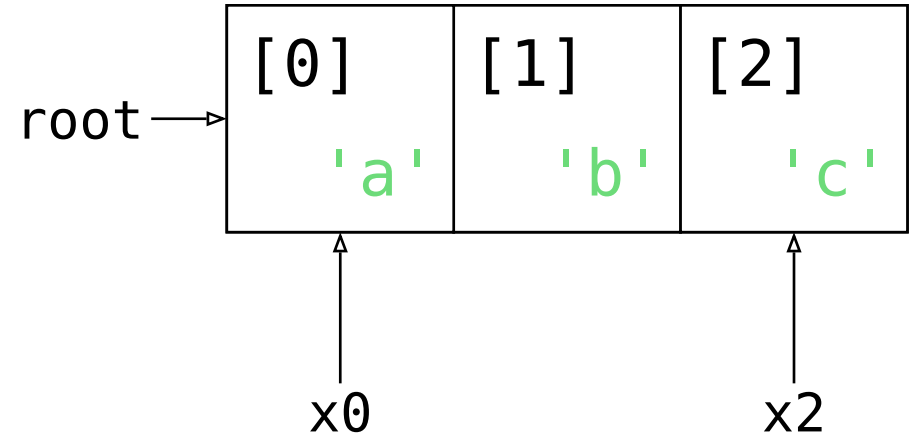
```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```



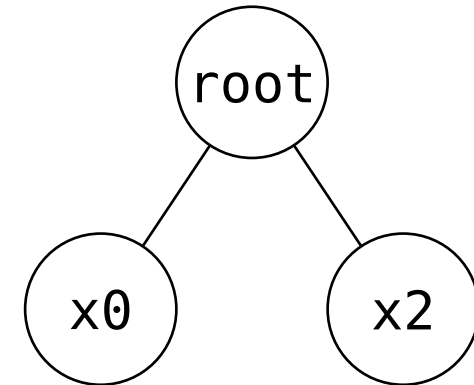
Desired outcome: not UB



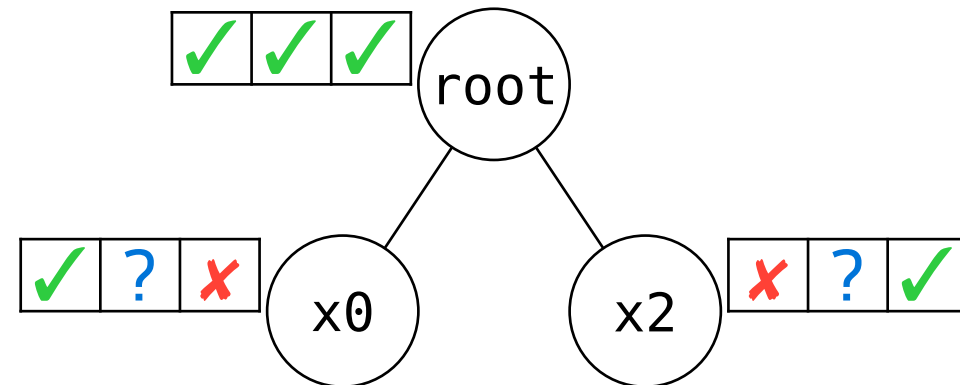
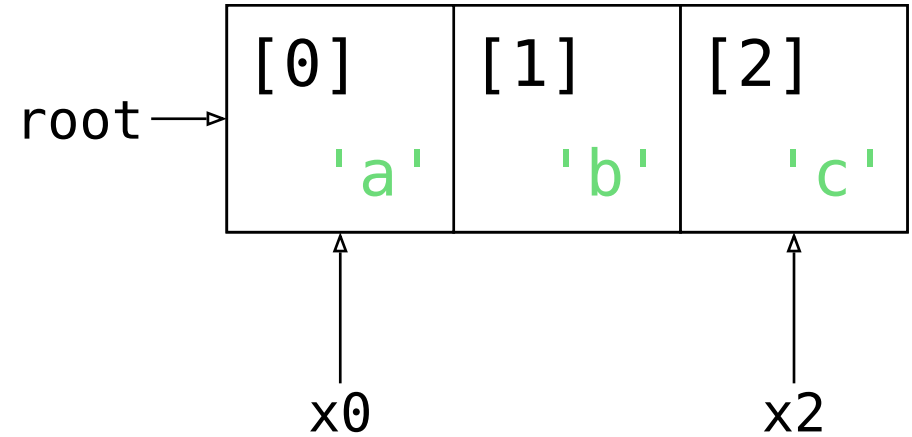
```
let mut root = vec!['a', 'b', 'c'];  
let x0 = &raw mut root[0];  
let x2 = &raw mut root[2];
```



Desired outcome: not UB




```
let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
```



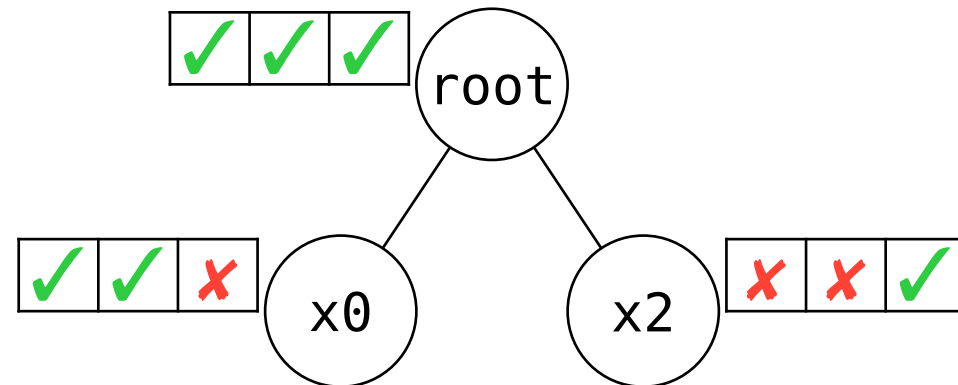
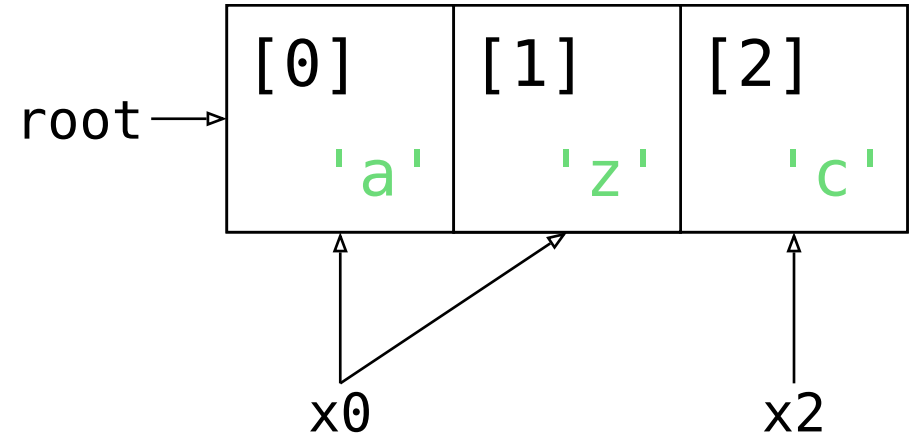
Desired outcome: not UB

```
let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
```

// Scenario 1

```
unsafe { *x0.add(1) = 'z' };
```

Desired outcome: not UB

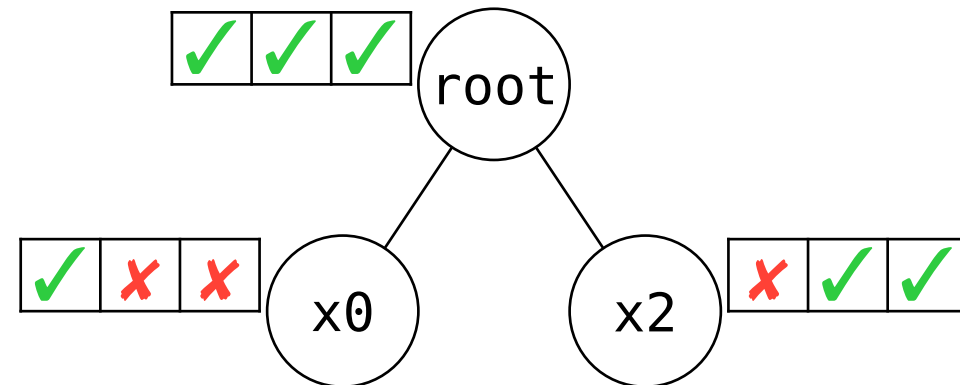
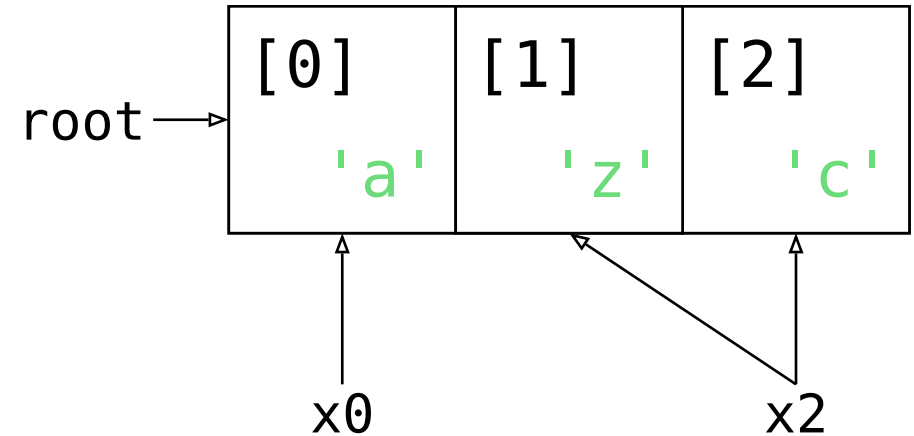


```
let mut root = vec!['a', 'b', 'c'];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
```

// Scenario 2

```
unsafe { *x2.sub(1) = 'z'; }
```

Desired outcome: not UB



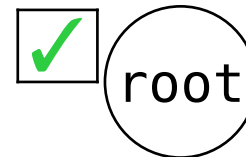
A second look at the motivating example

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

Desired outcome: UB

A second look at the motivating example

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

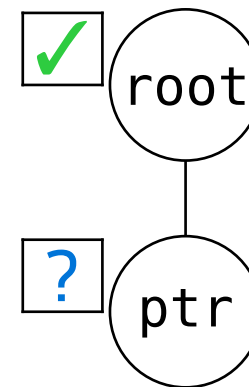


Desired outcome: UB

A second look at the motivating example

From Stacks to Trees

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

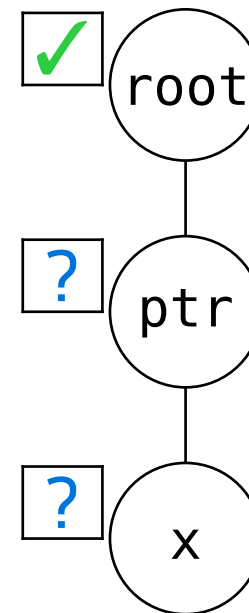


Desired outcome: UB

A second look at the motivating example

From Stacks to Trees

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

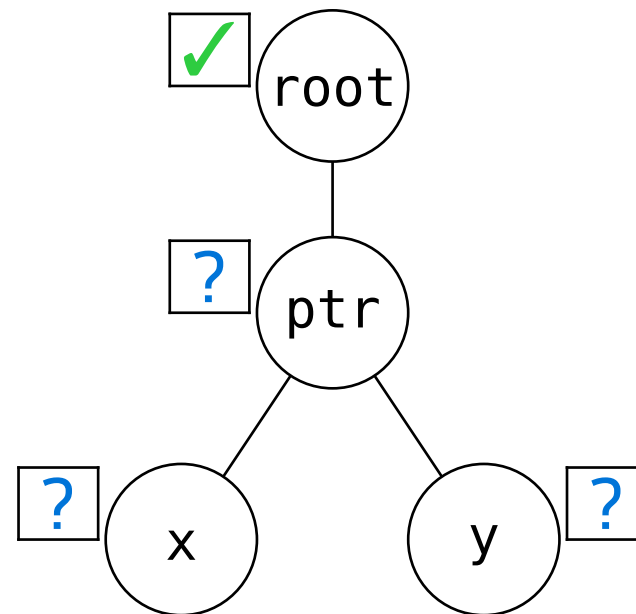


Desired outcome: UB

A second look at the motivating example

From Stacks to Trees

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

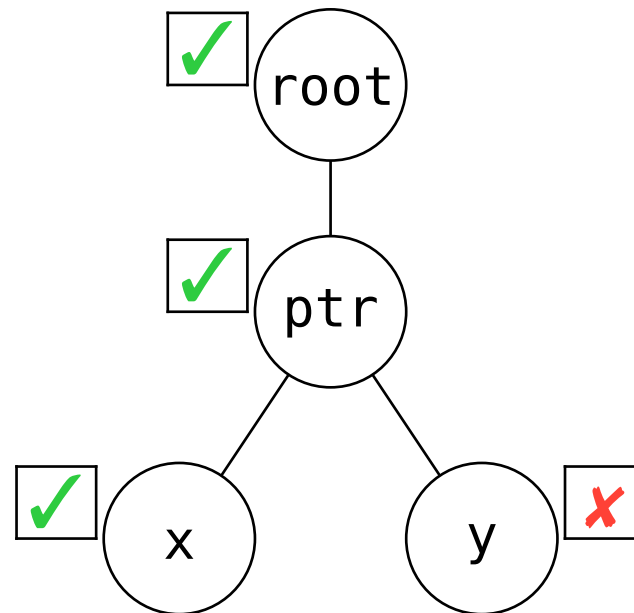


Desired outcome: UB

A second look at the motivating example

From Stacks to Trees

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```



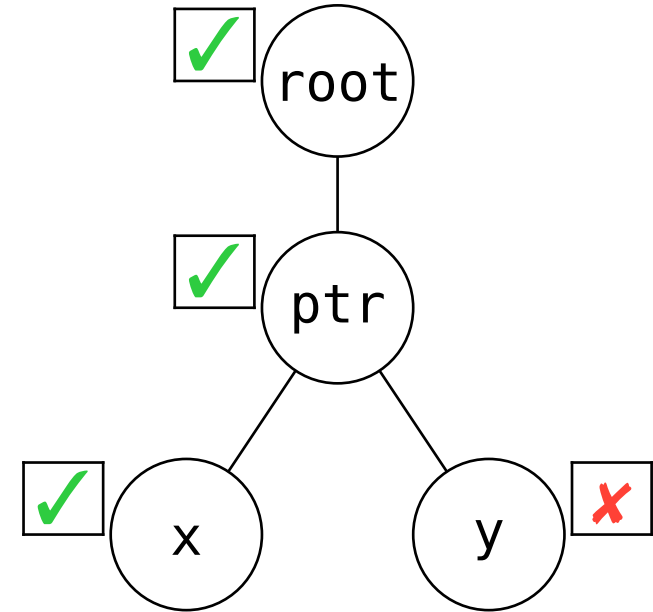
Desired outcome: UB

A second look at the motivating example

From Stacks to Trees

```
let mut root = 42;  
let ptr = &raw mut root;  
let x = unsafe { &mut *ptr };  
let y = unsafe { &mut *ptr };  
// inline write_both(x, y):  
*x = 13;  
*y = 20;
```

UB!



Desired outcome: UB

Evaluation

TB should enable desired optimizations

i.e. have enough UB to rule out problematic patterns

- formalized in Rocq (+Simuliris)
- a selection of optimizations proven
 - ✓ delete read through `&mut` or `&`
 - ✓ insert read through `&` in function
 - ✓ move read down for `&mut` or `&` in function
- ...

It should be possible to write **unsafe** code free of UB Evaluation

i.e. UB should be predictable and not too common

54% fewer tests have aliasing UB according to Tree Borrows

Only 31 ($< 0.01\%$) tests are regressions, all easily fixable.

(Out of 30 000 libraries, 400 000+ working tests)

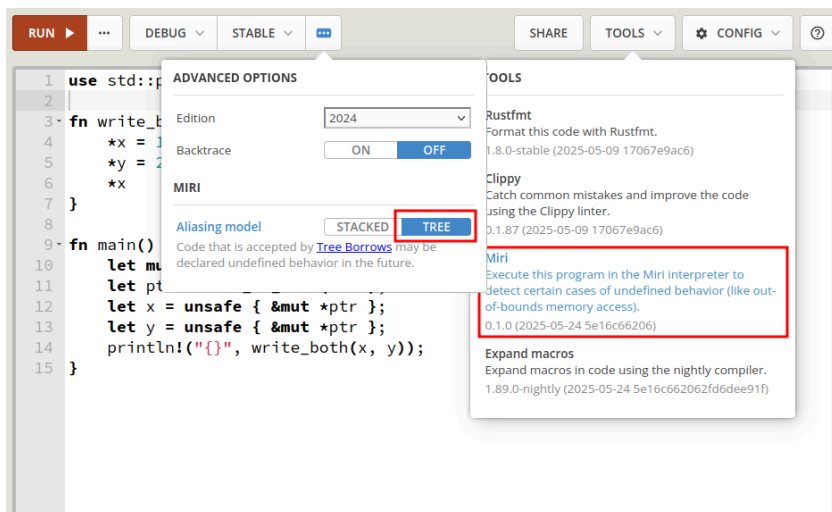
“Tree Borrows accepts more real-world programs that call foreign functions than Stacked Borrows due to differences in handling pointer arithmetic.”

A Study of Undefined Behavior Across Foreign Function Boundaries in Rust Libraries,
by I. McCormack, J. Sunshine, J. Aldrich @ ICSE'25

Conclusion

Try it out: supported by Miri

Also on the Rust Playground
(play.rust-lang.org)



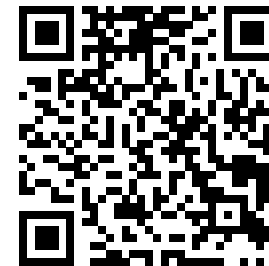
Postdoc positions available

@ ETH Zurich ralf.jung@inf.ethz.ch

@ MPI-SWS dreyer@mpi-sws.org

Learn more:

[plf.inf.ethz.ch/research/
pldi25-tree-borrows.html](https://plf.inf.ethz.ch/research/pldi25-tree-borrows.html)



Includes e.g. handling of raw pointers and interior mutability.